Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 5

Disciplina Tehnici de programare

Tema Funcții metode și importuri pe web

A efectuat st. gr. SI-263 Bacalu Silviu Laurențiu

A verificat asistent universitar Reițman Patricia

Chișinău 2026

SARCINA LUCRĂRII

Scopul lucrării este însușirea importurilor, metodelor și funcțiilor Python prin comunicarea controlată cu un site web real. Exercițiile urmăresc trecerea de la utilizarea bibliotecilor existente la scrierea unui modul propriu capabil să genereze un raport tehnic despre un site.

- importarea modulelor din biblioteca standard și a pachetelor externe instalate cu pip
- utilizarea atributelor și metodelor obiectelor Response, str, dict și Session
- definirea funcțiilor cu parametri, valori implicite, valori returnate, docstring și adnotări de tip
- tratarea excepțiilor produse de conexiune, codurile HTTP și datele invalide
- prelucrarea URL-urilor, HTML-ului, fișierelor JSON și CSV, DNS-ului și certificatelor TLS
- organizarea funcțiilor într-un modul propriu și executarea programului prin argumente din linia de comandă
- generarea raportului final report.json pentru domeniul https://cybercor.org

Structura exercițiilor

Tabelul 1.1 Structura exercițiilor realizate

Partea	Conținut	Exerciții	Fișier principal
1	Importuri	1–8	part1_imports.py
2	Metode și atribute web	9–20	part2_methods.py
3	Funcții proprii	21–34	part3_functions.py
4	Module standard	35–43	part4_modules.py
5	Modulul propriu	44–50	webtools.py și main.py

Mediul de lucru

Lucrarea a fost realizată în Windows, folosind Python 3.14.7 și Visual Studio Code. Biblioteca externă requests a fost instalată prin pip. Proiectul a fost organizat în directorul web_lab, iar cererile de citire au vizat domeniul cybercor.org. Serviciul httpbin.org a fost folosit pentru testele care necesitau antete personalizate.

Fiecare cerere de rețea folosește o limită de timp, iar cererile succesive sunt separate prin pauze de o secundă. Aceste reguli reduc riscul blocării programului și evită trimiterea rapidă a unui număr mare de cereri către server.

Noțiuni teoretice

Tabelul 1.2 Noțiuni teoretice utilizate

Noțiune	Rol în laborator	Exemplu
Import	Aduce în program cod dintr-un modul sau pachet	import requests
Atribut	Păstrează o valoare asociată unui obiect	response.status_code
Metodă	Execută o operație asupra unui obiect	response.json()
Funcție	Grupează cod reutilizabil și poate returna o valoare	get_title(html)
Modul	Fișier Python care poate fi importat	webtools.py

1 IMPORTURI

Prima parte prezintă diferența dintre biblioteca standard și pachetele externe. Exercițiile arată mai multe forme de import și modul în care documentația unui pachet poate fi consultată direct din Python.

Tabelul 1.3 Rezolvarea exercițiilor 1–8

Ex	Implementare	Rezultat și observație
1	Au fost importate requests și urllib.request. Versiunea requests a fost afișată, iar comentariul explică instalarea externă prin pip.	Modulul requests este extern, în timp ce urllib aparține bibliotecii standard.
2	Aceeași cerere a fost executată prin requests.get și prin numele get importat direct.	Ambele forme returnează un obiect Response; importul complet păstrează originea funcției vizibilă.
3	requests a fost importat ca rq și pagina a fost cerută cu rq.get.	Aliasul este util când abrevierea rămâne clară și constantă.
4	Pagina a fost descărcată cu urllib.request.urlopen, iar octeții au fost decodați în UTF-8.	Atributul status și primele 200 de caractere confirmă răspunsul.
5	dir(requests) a afișat membrii pachetului.	get este funcție, Session este clasă, iar utils este modul.
6	help(requests.get) a fost folosit pentru identificarea parametrului timeout.	Cererea folosește timeout=TIMEOUT.
7	Durata totală a fost măsurată cu time.perf_counter și comparată cu response.elapsed.	Valorile apropiate confirmă timpul cererii, cu diferențe datorate operațiilor locale.
8	Importul bs4 a fost plasat în try și except ImportError.	Când pachetul lipsește, programul afișează comanda pip install beautifulsoup4.

Cod reprezentativ

```
import time
import requests
import urllib.request

TIMEOUT = 10
start = time.perf_counter()
response = requests.get(BASE_URL, timeout=TIMEOUT)
duration = time.perf_counter() - start
print(response.status_code, duration, response.elapsed.total_seconds())
```

```
PS C:\Universitate\TP\Laborator 5> python -u "c:\Universitate\TP\Laborator 5\part1_imports.py"
Versiunea requests: 2.34.2
Cu requests.get(): 200
Cu requests.get() (alias): 200
Traceback (most recent call last):
  File "c:\Universitate\TP\Laborator 5\part1_imports.py", line 34, in <module>
    with urllib.request.urlopen(BASE_URL, timeout=TIMEOUT) as page:
         ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 187, in urlopen
    return opener.open(url, data, timeout)
           ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 493, in open
    response = meth(req, response)
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 602, in http_response
    response = self.parent.error(
        'http', request, response, code, msg, hdrs)
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 531, in error
    return self._call_chain(*args)
           ~~~~~~~~~~~~~~~~^^^^^^^
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 464, in _call_chain
    result = func(*args)
  File "C:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\urllib\request.py", line 611, in http_error_default
    raise HTTPError(req.full_url, code, msg, hdrs, fp)
urllib.error.HTTPError: HTTP Error 403: Forbidden
PS C:\Universitate\TP\Laborator 5>
```

Figura 1.1 Prima execuție și tratarea răspunsului HTTP 403

```
PS C:\Universitate\TP\Laborator 5> python -u "c:\Universitate\TP\Laborator 5\part1_imports.py"
Versiunea requests: 2.34.2
Cu requests.get(): 200
Cu requests.get() (alias): 200
Status urllib: 403
Serverul a refuzat cererea.
error code: 1010

['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadTimeout', 'Request', 'RequestException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequired', '__all__', '__author__', '__author_email__', '__build__', '__builtins__', '__cached__', '__cake__', '__copyright__', '__description__', '__doc__', '__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__', '__url__', '__version__', '_check_cryptography', '_internal_utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', 'charset_normalizer_version', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post', 'put', 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils', 'warnings']
Help on function get in module requests.api:

get(
    url: _t.UriType,
    params: _t.ParamsType = None,
    **kwargs: Unpack[_t.GetKwargs]
) -> Response
    Sends a GET request.

-- More  --
```

Figura 1.2 Verificarea importurilor și obținerea codului HTTP 200

```
    ) -> Response
        Sends a GET request.


    Status cu timeout: 200
    Durata totala: 0.7652744999995775 secunde
    response.elapsed: 0.761822 secunde
    instalati modulul cu: pip install beautifulsoup4
    PS C:\Universitate\TP\Laborator 5>
```

Figura 1.3 Cronometrarea cererii și mesajul pentru modulul bs4

2 METODE ȘI ATRIBUTE WEB

Obiectul Response furnizat de requests conține informațiile primite de la server. Atributele se citesc direct, iar metodele execută operații, precum verificarea codului de stare, conversia JSON sau prelucrarea șirurilor.

Tabelul 2.1 Rezolvarea exercițiilor 9–20

Ex	Implementare	Rezultat și observație
9	Au fost afișate status_code, ok, url și encoding.	Toate sunt atribute deoarece se citesc fără paranteze.
10	raise_for_status a verificat pagina principală și o adresă inexistentă.	requests.HTTPError a fost prins și transformat într-un mesaj clar.
11	headers.items a permis parcurgerea antetelor ca perechi nume–valoare.	Fiecare antet a fost afișat pe o linie.
12	headers.get a citit Server și Content-Type cu o valoare implicită.	Numele antetelor HTTP nu depind de majuscule.
13	response.text.lower().count('cyber') a numărat aparițiile cuvântului.	Metodele pot fi înlănțuite deoarece lower returnează un nou șir.
14	find, len, slicing și strip au extras conținutul dintre tagurile title.	Titlul este obținut fără biblioteci HTML suplimentare.
15	splitlines a separat HTML-ul, iar max cu key=len a găsit linia cea mai lungă.	Au fost afișate numărul de linii și lungimea maximă.
16	startswith('https://') a verificat schema URL-ului final.	Programul a raportat o conexiune securizată.
17	response.history a fost parcurs pentru cererea pornită de la HTTP.	Au fost observate codurile și adresele intermediare până la URL-ul final.
18	Au fost comparate cererile HEAD și GET prin lungimea response.content.	HEAD întoarce antete fără corpul complet al paginii.
19	Cookie-urile au fost parcurse și au fost citite name și secure.	În lipsa lor, programul afișează un mesaj explicit.
20	O sesiune requests.Session a trimis un User-Agent către httpbin.org/headers.	Răspunsul JSON a confirmat antetul personalizat.

Prelucrarea răspunsului

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code, response.ok, response.url, response.encoding)

for name, value in response.headers.items():
    print(f"{name}: {value}")

count = response.text.lower().count("cyber")
secure = response.url.startswith("https://")
```

În această etapă am observat clar diferența dintre response.text, care este un atribut de tip str, și response.json(), care este o metodă ce încearcă să transforme corpul răspunsului în obiecte Python.

3 FUNCȚII PROPRII

Funcțiile din această parte separă cererile de rețea de analiza datelor și reduc repetarea codului. Fiecare funcție are o sarcină clară, tipuri indicate și un rezultat care poate fi refolosit de alte funcții.

Tabelul 3.1 Rezolvarea exercițiilor 21–34

Ex	Implementare	Rezultat și observație

Ex	Implementare	Rezultat și observație
21	fetch(url) returnează obiectul Response.	Codul de stare poate fi citit după apel.
22	get_status(url) returnează numai codul HTTP.	Funcția a verificat /, /robots.txt și /sitemap.xml cu pauze între cereri.
23	fetch a primit parametrul implicit timeout=10.	Au fost testate valoarea implicită și timeout=3.
24	get_title(html) separă descărcarea de analiza conținutului.	Funcția întoarce titlul sau None.
25	Funcției get_title i-a fost adăugat un docstring.	help(get_title) afișează scopul funcției.
26	Au fost adăugate adnotări de tip, apoi get_status(123) a fost testată.	Adnotările descriu tipurile, dar nu blochează automat un argument greșit.
27	page_exists tratează requests.RequestException.	URL-urile invalide întorc False în loc să oprească programul.
28	check_paths construiește un dicționar cale–cod.	Rezultatele mai multor resurse sunt returnate împreună.
29	get_header acceptă name și default.	Apelul cu name='Server' arată folosirea argumentelor cu nume.
30	security_headers verifică cinci antete importante.	Dicționarul rezultat conține valori bool.
31	score_headers numără valorile True.	În execuția documentată scorul obținut a fost 3/5.
32	fetch_robots descarcă robots.txt, iar disallowed_paths extrage directivele Disallow.	Funcția acceptă și None; pentru test au rezultat liste goale.
33	response_times(*urls) măsoară mai multe adrese primite pozițional.	Rezultatul este un dicționar URL–secunde.
34	log(message, **details) colectează detalii cu nume.	Mesajul și perechile cheie–valoare sunt afișate pe aceeași linie.

Extragerea titlului și verificarea securității

```
def get_title(html: str) -> str | None:
    start = html.lower().find("<title>")
    if start == -1:
        return None
    start += len("<title>")
    end = html.lower().find("</title>", start)
    if end == -1:
        return None
    return html[start:end].strip()

def score_headers(results: dict[str, bool]) -> str:
    return f"{sum(results.values())}/{len(results)}"
```

```python
# exercitiu 23

def get_title(html):
    """Returnează titlul paginii din conținutul HTML"""
    start = html.find("<title>")

    if start == -1:
        return None

    start += len("<title>")
    end = html.find("</title>", start)

    if end == -1:
        return None

    return html[start:end].strip()

title = get_title(default_response.text)
print("\nTitlul paginii principale:", title)

# exercitiu 24
def get_title(html: str) -> str | None:
```

Figura 3.1 Implementarea funcției get_title în fișierul part3_functions.py

```
Problems  Output  Debug Console  Terminal  Ports

Scorul antetelor de securitate: 3/5
Traceback (most recent call last):
  File "c:\Universitate\TP\Laborator 5\part3_functions.py", line 217, in <module>
    print("\nCai interzise:", disallowed_paths(robots_text))
                             ~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^
  File "c:\Universitate\TP\Laborator 5\part3_functions.py", line 202, in disallowed_paths
    for line in robots_text.spitlines():
                ^^^^^^^^^^^^^^^^^^^^
AttributeError: 'str' object has no attribute 'spitlines'. Did you mean: 'splitlines'?
PS C:\Universitate\TP\Laborator 5>
```

Figura 3.2 Eroarea produsă de apelul greșit spitlines

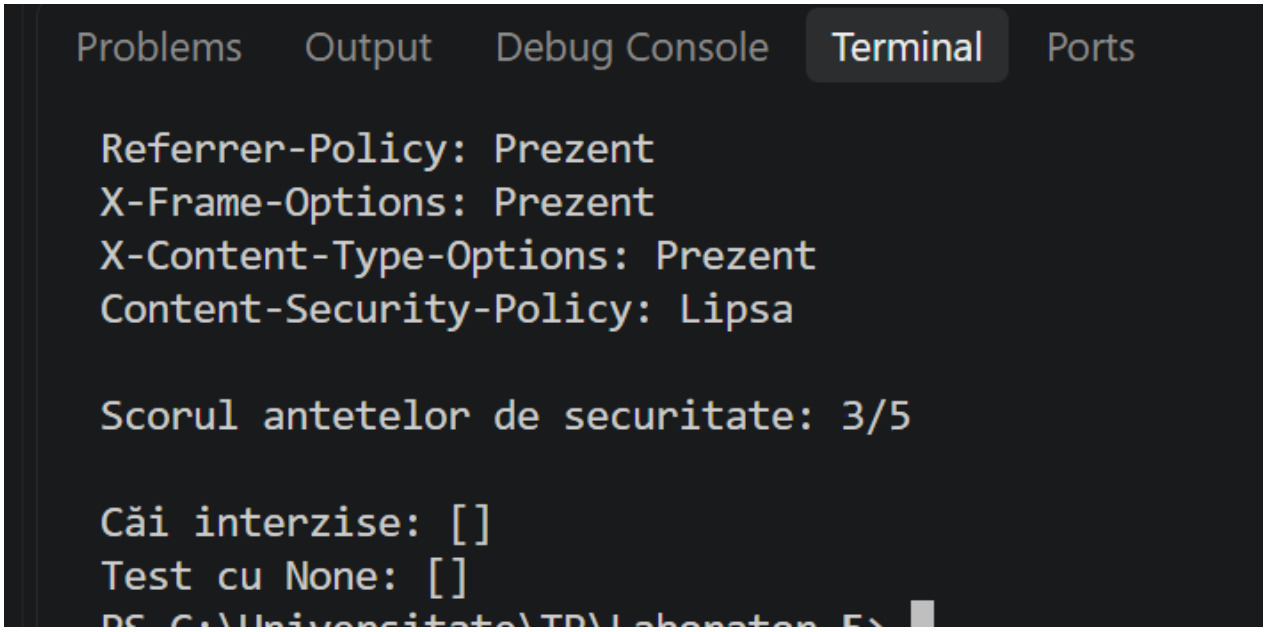


Figura 3.3 Rezultatul după corectarea splitlines și calcularea scorului 3 din 5

4 MODULE DIN BIBLIOTECA STANDARD

Partea a patra combină urllib.parse, re, html.parser, hashlib, json, socket și ssl. Aceste module permit analiza structurii unui URL, extragerea informațiilor din HTML și colectarea datelor tehnice despre domeniu.

Tabelul 4.1 Rezolvarea exercițiilor 35–43

Ex	Implementare	Rezultat și observație
35	urlparse a separat schema, domeniul, calea, interogarea și fragmentul.	Fiecare componentă a URL-ului poate fi prelucrată independent.
36	urljoin a transformat căile relative în URL-uri absolute.	Au fost tratate /about, contact.html și ../index.html.
37	re.findall a extras valorile href, iar dict.fromkeys a eliminat duplicatele.	Funcția extract_links returnează lista în ordinea apariției.
38	split_links a combinat urljoin cu urlparse.	Legăturile au fost separate în liste interne și externe.
39	ImageFinder a moștenit HTMLParser și a suprascris handle_starttag.	Testul assert confirmă colectarea atributelor src din tagurile img.
40	page_fingerprint calculează SHA-256 pentru response.content.	Amprenta se modifică atunci când se modifică octeții paginii.
41	Antetele au fost salvate în headers.json și citite cu json.load.	Valoarea Content-Type a fost regăsită după normalizarea cheilor.
42	socket.gethostbyname a rezolvat domeniul cybercor.org.	Adresa IPv4 depinde de răspunsul DNS din momentul execuției.
43	socket și ssl au deschis conexiunea TLS și au citit notAfter.	Funcția întoarce numărul întreg de zile până la expirare.

Parserul de imagini

```
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)
```

Interogarea DNS și certificatul TLS

```
def resolve(hostname: str) -> str:
    return socket.gethostbyname(hostname)

def cert_days_left(hostname: str) -> int:
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as secure_sock:
            certificate = secure_sock.getpeercert()
    expires_at = ssl.cert_time_to_seconds(certificate["notAfter"])
    return int((expires_at - datetime.now(timezone.utc).timestamp()) // 86400)
```

5 CONSTRUIREA MODULULUI PROPRIU

Ultima parte reorganizează funcțiile într-un modul reutilizabil. webtools.py conține logica, iar main.py citește argumentul din linia de comandă, generează fișierele CSV și JSON și afișează rezultatele.

Tabelul 5.1 Rezolvarea exercițiilor 44–50

Ex	Implementare	Rezultat și observație
44	Funcțiile reutilizabile au fost mutate în webtools.py.	main.py le poate apela prin prefixul webtools.
45	get_title și security_headers au fost importate selectiv.	O definiție locală cu același nume ar înlocui numele importat, dar webtools.get_title ar rămâne disponibil.
46	Blocul if __name__ == '__main__' execută autotestul numai la rularea directă.	La import, __name__ conține numele modulului și autotestul nu pornește.
47	DEFAULT_HEADERS păstrează User-Agent-ul laboratorului.	Constanta este importată și afișată din main.py.
48	argparse citește URL-ul din linia de comandă.	Comanda corectă este python main.py https://cybercor.org.
49	Modulul csv salvează path, status și checked_at.	Fișierul report.csv conține verificarea celor trei căi.
50	site_report colectează datele și salvează report.json.	Raportul include starea, titlul, IP-ul, redirecționările, securitatea, certificatul, legăturile și robots.txt.

Structura proiectului

```
web_lab/
├── webtools.py
├── main.py
├── report.csv
└── report.json
```

Citirea argumentului și pornirea programului

```
parser = argparse.ArgumentParser(description="Raport despre site-ul laboratorului.")
parser.add_argument("url", help="Folosește https://cybercor.org")
args = parser.parse_args()

if __name__ == "__main__":
    main()
```

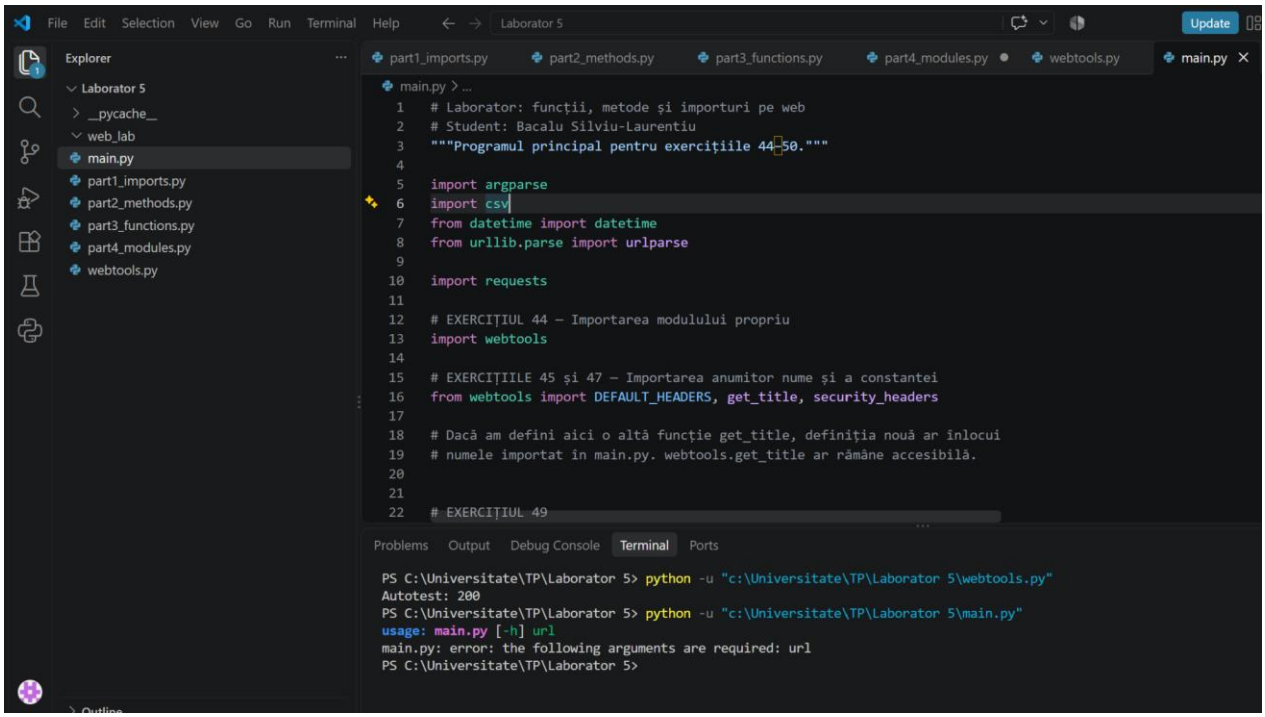


Figura 5.1 Structura proiectului și mesajul argparse când argumentul URL lipsește

Mesajul din figură nu indică o eroare în implementare. Argumentul url este pozițional și obligatoriu, de aceea programul trebuie pornit cu comanda python main.py https://cybercor.org. Rularea directă a fișierului webtools.py a executat autotestul și a returnat codul 200.

Funcția raportului final

```
def site_report(url: str) -> dict:
    response = fetch(url)
    header_results = security_headers(url)
    internal, external = split_links(extract_links(response.text), response.url)
    report = {
        "status_code": response.status_code,
        "title": get_title(response.text),
        "ip": resolve(urlparse(url).hostname),
        "security_score": score_headers(header_results),
        "internal_links_count": len(internal),
        "external_links_count": len(external),
        "disallowed_paths": disallowed_paths(fetch_robots(url)),
    }
    with open("report.json", "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2, ensure_ascii=False)
    return report
```

6 DATE ȘI STRUCTURI FOLOSITE

Tabelul 6.1 Date și structuri folosite

Nume	Tip	Rol
BASE_URL	str	Adresa de bază https://cybercor.org

Nume	Tip	Rol
ECHO_URL	str	Adresa serviciului httpbin folosit la testele de trimitere
TIMEOUT	int	Numărul maxim de secunde acordat unei cereri
DEFAULT_HEADERS	dict str str	Antetul User-Agent al modulului propriu
SECURITY_HEADERS	list str	Lista celor cinci antete verificate
response	requests.Response	Răspunsul primit de la server
internal external	list str	Legăturile clasificate după domeniu
report	dict	Datele agregate înainte de salvarea în report.json
errors	dict	Erorile reținute fără oprirea raportului complet

7 DEPANAREA PROGRAMULUI

În timpul lucrului au apărut atât erori reale de cod, cât și mesaje normale produse de validarea argumentelor. Păstrarea lor în raport arată modul în care cauza a fost identificată și corectată.

Tabelul 7.1 Sinteza erorilor întâlnite

Situație observată	Cauză	Corecție aplicată
HTTPError 403 Forbidden	Serverul a refuzat una dintre cererile inițiale	Excepția a fost tratată, iar cererea a fost repetată controlat; execuțiile următoare au primit 200
ModuleNotFoundError pentru bs4	Pachetul BeautifulSoup nu era instalat	A fost afișată comanda pip install beautifulsoup4
AttributeError pentru spitlines	Numele metodei a fost scris greșit	spitlines a fost înlocuit cu splitlines
internal_links sau external_links nedefinite	Listele erau folosite înainte de atribuirea rezultatului split_links	A fost adăugată atribuirea internal_links, external_links = split_links(...)
Argumentul url este obligatoriu	main.py a fost pornit fără parametrul pozițional	Programul a fost rulat ca python main.py https://cybercor.org

8 REZULTATUL FINAL

Proiectul final validează adresa primită, colectează informațiile disponibile și reține separat erorile fiecărei verificări. Astfel, o problemă DNS, TLS sau HTTP nu împiedică salvarea celorlalte rezultate.

Tabelul 8.1 Fișierele generate de program

Fișier	Conținut
report.csv	Calea verificată, codul de stare și data în format ISO
report.json	URL, status, titlu, IP, redirecționări, antete, certificat, legături, robots.txt și erori

```
python main.py https://cybercor.org
```

Aceasta este comanda finală de rulare. Valorile pentru IP, durata certificatului și numărul de legături pot varia deoarece sunt citite de pe site în momentul execuției.

9 FRAGMENTE DIN CODUL FINAL

Fragmentele următoare prezintă organizarea fișierului main.py. Codul complet rămâne în directorul proiectului, împreună cu modulul webtools.py.

```
# Laborator: funcții, metode și importuri pe web
# Student: Bacalu Silviu-Laurentiu
"""Programul principal pentru exercițiile 44–50."""

import argparse
import csv
from datetime import datetime
from urllib.parse import urlparse

import requests

# EXERCIȚIUL 44 — Importarea modulului propriu
import webtools

# EXERCIȚIILE 45 și 47 — Importarea anumitor nume și a constantei
from webtools import DEFAULT_HEADERS, get_title, security_headers

# Dacă am defini aici o altă funcție get_title, definiția nouă ar înlocui
# numele importat în main.py. webtools.get_title ar rămâne accesibilă.


# EXERCIȚIUL 49
def save_csv(base: str) -> None:
    """Salvează verificarea celor trei căi în report.csv."""
    results = webtools.check_paths(base, ["/", "/robots.txt", "/sitemap.xml"])
    with open("report.csv", "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["path", "status", "checked_at"])
        for path, status in results.items():
            writer.writerow([path, status if status is not None else "eroare", datetime.now().isoformat()])
    print("Salvat în report.csv")


def main() -> None:
    """Citește URL-ul și execută demonstrațiile și raportul final."""
    # EXERCIȚIUL 48 — Argument pozițional din linia de comandă
    parser = argparse.ArgumentParser(description="Raport despre site-ul laboratorului.")
    parser.add_argument("url", help="Folosește https://cybercor.org")
    args = parser.parse_args()
    parsed = urlparse(args.url)
    if parsed.scheme not in ("http", "https") or parsed.hostname != "cybercor.org" or parsed.path not in
("", "/") or parsed.query or parsed.fragment or parsed.port is not None or parsed.username is not None:
        parser.error("Pentru acest laborator folosește https://cybercor.org sau http://cybercor.org.")

    print("Exercițiul 47 — DEFAULT_HEADERS:", DEFAULT_HEADERS)

    try:
        # EXERCIȚIUL 44 — Apel prin numele modulului
        response = webtools.fetch(args.url)
        print("Exercițiul 44 — Titlu:", webtools.get_title(response.text))

        # EXERCIȚIUL 45 — Apel prin numele importate
        print("Exercițiul 45 — Titlu:", get_title(response.text))
        print("Antete:", security_headers(args.url))
    except requests.RequestException as error:
        print("Demonstrația inițială a eșuat:", error)

    base = f"{parsed.scheme}://{parsed.netloc}"
    save_csv(base)
    webtools.site_report(args.url)


if __name__ == "__main__":
    main()
```

CONCLUZII

În această lucrare am folosit Python pentru a comunica în mod controlat cu un site web și pentru a transforma răspunsurile HTTP în informații ușor de analizat. Exercițiile au clarificat diferența dintre importuri, atribute, metode și funcții, precum și rolul valorilor returnate în reutilizarea codului.

Am separat descărcarea paginii de prelucrarea HTML-ului, am adăugat adnotări de tip și am tratat excepțiile produse de conexiuni sau adrese invalide. Funcțiile au fost apoi combinate pentru verificarea antetelor de securitate, citirea fișierului robots.txt, extragerea legăturilor, rezolvarea DNS și citirea certificatului TLS.

Erorile întâlnite au evidențiat importanța numelor exacte ale metodelor și a ordinii în care sunt definite variabilele. Corectarea metodei splitlines, inițializarea listelor de legături și transmiterea argumentului url au permis finalizarea programului fără erori de sintaxă.

Rezultatul final este un modul propriu, webtools.py, importat din main.py. Programul poate fi lansat din terminal pentru domeniul indicat și generează două fișiere de raport. Această organizare face codul mai clar, mai ușor de testat și mai simplu de extins.

Competențe dobândite

- utilizarea importurilor complete, selective și cu alias
- citirea atributelor și apelarea metodelor obiectelor web
- scrierea funcțiilor reutilizabile și documentate
- tratarea excepțiilor și interpretarea mesajelor de eroare
- prelucrarea URL-urilor, HTML-ului, JSON-ului și CSV-ului
- organizarea unui proiect Python în module proprii

BIBLIOGRAFIE

1. Python Software Foundation. The Python Tutorial. https://docs.python.org/3/tutorial/ accesat la 07.10.2026.
2. Python Software Foundation. Python Standard Library. https://docs.python.org/3/library/ accesat la 07.10.2026.
3. Requests documentation. https://requests.readthedocs.io/ accesat la 07.10.2026.
4. Laborator Python funcții metode și importuri pe web. Material didactic pentru exercițiile 1–50.
5. Capturile de ecran realizate în timpul implementării și depanării laboratorului la 06–07.10.2026.